

Diagnosing Domain Mismatch in LLM Security Guardrails: A Deterministic Schema Check for SOC Alert Triage

Asma*

*TECIP, Scuola Superiore Sant’Anna
CNIT/PNTLab Pisa
Pisa, Italy

Abstract—Security operations centers increasingly rely on large language models (LLMs) to triage the alert volumes that outpace manual review, but LLM-based triage pipelines inherit LLM-specific risks such as prompt injection through untrusted alert fields. A natural mitigation is a learned guardrail that screens alert text before it reaches the model. We report a case study in which exactly this approach failed silently on real data, why it failed, and what replaced it. A TF-IDF and logistic-regression classifier trained to detect prompt-injection attempts in an alert-title field scored 0.883 F1 on a chat-style jailbreak benchmark, then achieved only 52.5% best-case accuracy when evaluated against real Security Incident Prediction (GUIDE) data. We show the root cause is not a training-data mismatch in style, as first suspected, but a schema mismatch: the target field is a numeric identifier in the real dataset, not free text at all, so no amount of domain-matched fine-tuning could have closed the gap. We replace the learned classifier with a deterministic type check that separates the same two classes by construction, verify it against the field’s full value space (all 9,516,837 rows of the real training data, zero false positives), and report its cost (microsecond-scale) against the classifier it replaces. We also report an unrelated but related reliability finding: a later routing change silently disconnected part of the pipeline’s graph, producing a hardcoded default verdict for every alert with no runtime error, caught only by a post-hoc audit – and describe the structural regression tests now guarding against a recurrence. We evaluate the full triage agent against a Random Forest baseline on real GUIDE data and report the result honestly: the LLM/RF-fallback hybrid does not outperform the RF baseline’s raw classification metrics on the largest sample we ran (macro F1 0.669 vs. 0.751, $n = 999$), a result we discuss rather than omit.

Index Terms—security operations center, large language models, prompt injection, guardrails, alert triage, MITRE ATT&CK, negative results

I. INTRODUCTION

Security operations center (SOC) analysts triage large volumes of alerts under time pressure, and most alerts turn out to be benign [8]. LLM-assisted triage – using a language model to read an alert, pull in relevant context, and produce a plain-language verdict with reasoning – has been shown to reduce analyst workload in both industry deployments [1] and recent research systems [4]. Because these systems put an LLM in

the path of attacker-influenced input (alert titles, device names, and other fields that may originate from a compromised or adversarial source), they inherit LLM-specific risks: prompt injection, where crafted text in an alert field attempts to override the model’s instructions rather than describe a genuine security event [3].

A standard mitigation is to add a guardrail stage before the LLM sees untrusted text – either deterministic pattern matching or a learned classifier. This paper documents a case where we built and evaluated both, expected the learned classifier to be the stronger of the two, and found instead that it had no usable signal on real production data. The reason turned out not to be what we first assumed. We present this as a full account, including the incorrect first hypothesis, because the eventual root cause – a schema property of the underlying dataset, not a text-domain gap – is the kind of finding that a purely metric-driven evaluation would not have surfaced on its own.

Our contributions are:

- A worked example of diagnosing *why* a security classifier fails on real data, going past accuracy numbers to the actual data schema, and showing the failure mode was structural (a numeric ID field, not text) rather than a distributional one (informal vs. formal language).
- A deterministic replacement guardrail that separates the same two classes by construction instead of by learned approximation, verified against the field’s entire real value space rather than a sample.
- An honest evaluation of the full triage pipeline against a classical baseline on real data, including a finding – a silent graph-wiring regression that made every alert default to the same verdict – that a purely accuracy-based evaluation had already missed once, and the structural tests we added to stop it recurring.
- A candid comparison of the LLM/RF-hybrid triage agent against the Random Forest baseline that does not outperform it on our largest real-data sample, discussed rather than omitted.

II. RELATED WORK

Industry SOC copilots such as Microsoft Security Copilot [1] demonstrate that LLMs integrated into SOC workflows reduce analyst triage time, but as a closed commercial system it offers no visibility into guardrail design or failure modes. Ferrag et al. [3] survey LLM applications across the cybersecurity lifecycle and separately catalog LLM-specific vulnerabilities including prompt injection, which motivates the guardrail design examined in this paper. CORTEX [4] argues for multi-agent, evidence-grounded triage over a single do-everything model, closely matching our own MITRE ATT&CK-grounded retrieval design (Section III); we adopt a hybrid LLM/classical-ML routing strategy rather than multi-agent decomposition, and this paper’s focus is the guardrail layer that sits in front of that routing rather than the routing architecture itself. Ndichu et al. [8] survey AI-driven alert screening broadly and review 22 alert-level datasets by representational gaps against real SOC environments; their taxonomy is the reference point we use in Section VI to discuss the limits of evaluating on a single static dataset. Ismail et al. [6] build a retrieval-augmented LLM copilot directly on the open-source Wazuh SIEM’s live alert stream – architecturally the closest system to ours, but grounded in a different (live, rule-engine-generated) alert source than the static dataset snapshot used here; we discuss this gap in Section VII. Red-team-side agentic tooling such as ADStrike [2] demonstrates the offensive counterpart of the agentic-AI pattern this paper applies defensively. GeNIS [7] addresses a labeled-data gap for small/medium-enterprise network traffic that our own dataset (Section V-A) does not cover, and is discussed as future work.

III. SYSTEM ARCHITECTURE

The triage pipeline is implemented as a directed graph (LangGraph) over a shared alert state. Given a raw alert, the pipeline: (1) builds a structured text context from the alert’s populated fields; (2) applies a deterministic regex-based input guardrail; (3) applies the schema guardrail described in Section IV; (4) retrieves MITRE ATT&CK technique context for any technique ID present on the alert; (5) routes the alert to either the LLM classifier or a Random-Forest fallback classifier, depending on how much analyst-readable evidence the alert carries; and (6) parses the resulting verdict, routing low-confidence results to a human-review checkpoint rather than auto-closing them. A failure or block at any guardrail stage routes directly to human review rather than reaching the LLM.

The routing decision at step 5 exists because a meaningful fraction of real GUIDE alerts carry almost no analyst-readable evidence: MITRE technique, suspicion level, and last-verdict fields are frequently absent, and AlertTitle/Category alone are numeric or closed-vocabulary codes rather than descriptive text (this is the same schema property discussed in Section IV-C). Alerts with fewer than two of these three discriminative fields populated are routed to the RF classifier instead of the LLM, on the reasoning that an LLM has little

basis for a per-alert judgment when the alert itself carries almost no readable signal.

IV. THE DOMAIN-MISMATCH INVESTIGATION

A. First guardrail: a learned classifier

The regex-based input guardrail (Section III) catches literal instruction-override and role-impersonation phrasing, but by construction cannot catch paraphrased or novel injection attempts. As a second-stage defense, we trained a TF-IDF + logistic-regression classifier on AlertTitle text, selected after benchmarking against Meta’s Prompt Guard and Protect AI’s LLM Guard on a chat-style jailbreak benchmark, where it scored 0.883 F1 – the strongest result of the three.

B. Real-data evaluation and a masking bug

Evaluated against real GUIDE alert data rather than the benchmark it was selected on, the classifier initially flagged 100% of real alerts as injection attempts. Investigating, we found an index bug in how the classifier’s output probability was read (`predict_proba(X)[0][0]`, the benign-class probability, was being used where `[0][1]`, the injection-class probability, was needed) – caught in code review before the initial finding was accepted at face value. After the fix, benign and injection scores on real alert text were statistically indistinguishable (mean 0.791 vs. 0.776); the best achievable accuracy across a threshold sweep was 52.5%, at chance level for a binary decision.

C. Root cause

The first hypothesis was a training-data style mismatch: the classifier had been benchmarked on conversational jailbreak text, and real SOC alert text might simply look different enough linguistically that the model’s learned features did not transfer. Before spending the following week’s time-boxed effort on a domain-matched fine-tune, we checked what AlertTitle actually contains in the real data. It is not text at all: across all 9,516,837 rows of the real training set, AlertTitle is a numeric ID field with 86,149 unique integer values (we re-verified this figure directly against the raw CSV during a later audit pass, confirming it independently rather than propagating it from the original finding). GUIDE’s own published description of the alert dataframe lists OrganizationId, DetectorId, ProductId, Category, and Severity as its only categorical columns – no free-text field exists in the schema at all. A TF-IDF classifier cannot find linguistic signal in a field that was never linguistic; no amount of domain-matched training data would have closed that gap, because the gap was never about domain.

D. Deterministic schema guardrail

Since AlertTitle and DetectorId are supposed to always be numeric IDs, the correct second-stage check is not a classifier but direct type validation: does the field parse as an integer. An injection payload is, by definition, not a valid integer, so this check separates the two classes by construction rather than by learned approximation, and costs microseconds

rather than a model load. We tested it two ways: on a synthetic 20-benign/20-injection set (100% accuracy, matching the earlier classifier’s benchmark-set performance but now on the real risk case rather than a proxy for it), and, during the audit for this paper, directly against the field’s entire real value space – all 9,516,837 `AlertTitle` values in the training set – confirming zero non-numeric values and therefore zero false positives across the complete population, not a sample of it. Unlike the learned classifier, this check was hard-gated into the pipeline: it has no threshold, no accuracy trade-off, and no probabilistic gray zone to be cautious about.

V. EVALUATION

A. Dataset

We use the Microsoft GUIDE dataset (CDLA-2.0), a public release of 1.6M real SOC alerts and 1M analyst-triaged incidents from over 6,100 organizations. All quantitative results in this paper were produced against the real dataset, not the schema-matched synthetic sample used for local development, and are reproducible from the committed artifacts in the project repository.

B. RF baseline

A Random Forest (200 trees) trained on 79,580 alerts (stratified 80/20 split, $n = 100,000$, fixed random seed) and evaluated on a 19,895-alert holdout achieves macro F1 0.751 (accuracy 0.772). We re-trained this baseline from scratch during the audit for this paper (rather than trusting the previously saved artifact) and reproduced macro F1 0.751 exactly.

C. Hybrid agent vs. RF baseline

On real GUIDE alerts, the full LLM/RF-hybrid pipeline was evaluated at two sample sizes. On a 300-alert sample, accuracy was 0.753 and macro F1 0.750 (81.3% of alerts routed to the RF fallback path, 18.7% to the LLM), closely matching the RF baseline. On a larger, later 999-alert sample – the largest real-data evaluation in the project, and the one we treat as primary here – accuracy was 0.674 and macro F1 0.669 (79.1% RF-routed, 20.9% LLM-routed), a roughly 8-point macro F1 gap below the RF baseline alone. We report both rather than the more favorable of the two: the smaller sample was the one originally written up, and the larger, more statistically robust sample was sitting in the repository’s committed results without ever having been discussed. We do not read this as a failure of the hybrid design so much as a reminder that raw classification metrics are not the only axis this system is designed against – the LLM path adds per-alert natural-language reasoning and MITRE-grounded context that the RF baseline does not produce at all, which the macro F1 number does not capture, but which is exactly the analyst-facing output the system exists to produce. We do not have an analyst-rated evaluation of that reasoning quality, and flag its absence as a limitation (Section VI) rather than assert the trade-off is worth it without evidence.

D. Guardrail cost and accuracy

The regex input guardrail runs in a mean 3.6 μ s per check (10,000-iteration microbenchmark), blocking 10,000/10,000 known injection strings with zero false positives on benign alerts in the same benchmark. The schema guardrail (Section IV) adds a second, equally cheap deterministic stage in front of the LLM/RF routing decision.

E. Throughput

On real-alert throughput benchmarks, the LLM path sustains roughly 0.37–0.57 alerts/second (mean latency 1.8–2.7 s per alert), dominated by external API latency and largely unaffected by added worker concurrency because the bottleneck is the remote provider, not local compute. The RF path, by contrast, sustains 16.7–41.9 alerts/second at the same prompt volume, scaling with local worker count as expected for CPU-bound inference. This is the practical justification for routing low-evidence alerts to RF rather than LLM: those alerts gain little from LLM reasoning (Section III) and the RF path is roughly two orders of magnitude faster.

F. Reliability engineering: a silent routing regression

A later change that introduced the schema guardrail into the pipeline graph also, unintentionally, deleted the conditional edge routing MITRE-enriched alerts onward to the LLM/RF classification step, without replacing it. The graph still compiled – the underlying graph library does not validate that every node has an outgoing edge – and ran to completion on every alert, but produced no verdict at all past that point. The evaluation harness’s own code, reading the missing verdict field with a default value, silently treated every alert as a low-confidence `FalsePositive` guess instead of raising an error. No evaluation had been re-run since the change landed, so nothing surfaced the regression until a manual, unrelated documentation audit found it days later. We root-caused it against the exact commit that introduced it, restored the missing edge, and added a structural regression test suite (the project’s first) that asserts every non-terminal node in the pipeline graph has a reachable outgoing edge, plus an end-to-end invocation test asserting a real verdict is produced. We consider this worth reporting alongside the classifier finding: both are cases where a plausible-looking result (a classifier’s benchmark score in one case, a batch of predictions in the other) was masking a structural problem that only became visible on inspecting the underlying mechanism rather than the output metric.

G. Adversarial robustness: a dynamic red-team evaluation

The evaluations above measure classification accuracy against labeled data, not whether an attacker-controlled alert field can manipulate the triage verdict itself. To test that question directly, we used the open-source `deepteam` framework [10] to dynamically generate adversarial inputs against `classify_with_llm` in isolation, bypassing both guardrails deliberately – they already have dedicated passing tests, and isolating the LLM node answers the narrower question of

whether the model itself is manipulable once adversarial content reaches it. Since deepteam has no built-in Groq provider, we implemented a Groq-backed judge/simulator wrapper, used as both roles. Three vulnerability types (hijacking, social-engineering goal theft, and cross-context indirect instruction) were crossed with four attack transforms (prompt injection, Base64, ROT13, and roleplay), yielding 12 test cases.

Of 12 cases, 7 completed and were conclusively scored, with a 0% attack success rate – every completed attack was resisted. The remaining 5 errored, concentrated entirely in the two attack methods (prompt injection, roleplay) whose generation step itself requires a structured JSON response from the judge model; the two deterministic transforms completed 3/3 each with zero errors. We traced the cause to the judge model producing malformed JSON and added a defensive JSON-repair layer to the wrapper. A same-day retry was blocked by the judge model’s daily token quota being exhausted from cumulative testing, leaving the fix unit-tested but not live-verified at the time.

We later live-verified the fix once the quota reset. A focused retry on the two problematic attack methods raised their conclusive rate from a combined 1/6 to 4/12 relative to the original run (prompt injection 0/3 $\rightarrow$ 1/6, roleplay 1/3 $\rightarrow$ 3/6), with all four newly-conclusive cases again resisting the attack. The fix is a real, measured improvement rather than a full resolution: the remaining cases still error on the same JSON-formatting failure mode, indicating the judge model’s structured-output reliability is a persistent limitation of this evaluation setup rather than a bug we introduced.

We also ran deepteam against the full defended pipeline, guardrails included, to test whether an attack that reaches the LLM is still caught downstream. This produced a counterintuitive result on inspection: 9 of 12 cases were conclusive with 0% attack success, but every conclusive case had actually been routed to the RF fallback classifier, not the LLM, because the synthesized test alert lacked the context fields (MITRE technique, suspicion level, prior verdict) that the pipeline’s own sparse-context routing logic (Section III) checks before choosing between the LLM and RF paths. The guardrails passed cleanly on every case; they simply were not the deciding factor, since the alert rarely reached the node they exist to protect. We report this as a test-harness limitation surfaced by inspecting per-case routing rather than trusting the summary pass rate – the same lesson as the routing regression above, that an aggregate metric in this codebase is not sufficient on its own to confirm a mechanism worked as intended.

VI. DISCUSSION AND LIMITATIONS

The schema guardrail’s guarantee is narrower than it may appear: it verifies a field is *an* integer, not that it is a *plausible* one. A crafted negative or out-of-range integer passes the check even though no real alert in the dataset has ever presented one, because the check’s only job – separating free text from IDs – does not require checking plausibility, and injection payloads are categorically excluded by the integer

check regardless. We note this as a documented scope boundary rather than an oversight: a different guardrail would be needed to catch a numeric-payload attack that this check was never designed to catch.

The RF baseline and fallback classifier were trained exclusively on GUIDE’s feature schema. We built (as part of this same research program, reported alongside this paper) a schema adapter translating alerts from Wazuh, an open-source SIEM, into the pipeline’s alert format, to test generalization beyond one static dataset snapshot. The LLM/guardrail path handles Wazuh-derived alerts correctly – confirmed by direct invocation, not merely a unit test asserting the mapping function’s output shape. The RF fallback path does not fail outright on Wazuh-derived alerts (the underlying classifier’s missing-value handling accepts the input and returns a normal-looking prediction), but a large majority of its expected feature columns are absent for a Wazuh-sourced alert – a sparsity pattern the model was never trained or validated against, unlike GUIDE’s own in-distribution missingness. We flag this explicitly as an open validity question rather than either fixing it speculatively or hiding it: routing a Wazuh-derived alert through the RF fallback currently produces a result of unknown reliability, and we do not report an accuracy number for it because we do not have labeled Wazuh data to check one against.

Evaluating against a single dataset snapshot – even a large, real one – limits how confidently any of these results generalize; Ndichu et al. [8]’s survey of 22 alert-level datasets is instructive here in cataloging just how differently existing benchmarks tend to be structured relative to live SOC environments. GeNIS [7], a 2025 dataset built specifically to address a labeled-data gap for small/medium-enterprise network traffic, is a candidate for extending this evaluation beyond GUIDE, but its flow-level schema does not map onto GUIDE’s incident/alert schema and would require its own loader rather than an extension of the existing one – noted as a scoping decision for future work rather than attempted here.

VII. FUTURE WORK

Four extensions follow directly from the limitations above. First, a live Wazuh deployment (rather than the schema-only adapter reported here) would let the pipeline’s guardrails and routing be tested against a continuously generated alert stream instead of a static snapshot, closer to the deployment described by Ismail et al. [6]. Second, integrating GeNIS [7] as a second evaluation dataset – once its flow-level schema is reconciled with an incident/alert-level loader – would test whether the RF baseline and schema guardrail’s strong GUIDE-specific results hold on a structurally different, SME-focused dataset. Third, the RF-fallback reliability gap on non-GUIDE alert sources (Section VI) motivates either labeled data collection on Wazuh-origin alerts specifically, or an origin-aware routing change that defaults non-GUIDE alerts to human review rather than the RF path until validated. Fourth, the full-graph red-team run (Section V-G) needs a

synthesized alert that carries at least one of the context fields `route_by_context` checks, so the attack payload actually reaches `classify_with_llm` instead of being diverted to the RF path before the guardrails' target is ever tested; the remaining unresolved prompt-injection/roleplay judge-model errors are a second, independent candidate for a stronger or differently-hosted judge model if one becomes available.

VIII. CONCLUSION

We diagnosed why a learned prompt-injection classifier had no usable signal on real SOC alert data: not a domain mismatch in text style, as first suspected, but a schema mismatch – the target field is a numeric ID, not text. We replaced it with a deterministic check verified against the field's complete real value space, at a fraction of the computational cost. We report this alongside an unrelated reliability finding from the same investigation – a silent routing regression masked by an evaluation harness's own default-value handling – because both point to the same lesson: inspecting the underlying data and control flow directly, rather than trusting an output metric, is what surfaced each problem. We evaluate the resulting pipeline against a classical baseline honestly, including a real-data comparison where the hybrid pipeline does not outperform the baseline on raw classification metrics, and discuss what that trade-off does and does not tell us.

ACKNOWLEDGMENT

REFERENCES

- [1] Microsoft, "Microsoft Security Copilot," Microsoft, 2024. [Online]. Available: <https://www.microsoft.com/en-us/security/business/ai-machine-learning/microsoft-copilot-for-security>
- [2] capture0x, "ADStrike: AI-Powered Modular Active Directory Red-Team Framework via MCP," GitHub repository, 2025. [Online]. Available: <https://github.com/capture0x/AdStrike>
- [3] M. A. Ferrag, F. Alwahedi, A. Battah, B. Cherif, A. Mechri, N. Tihanyi, T. Bisztray, and M. Debbah, "Generative AI in Cybersecurity: A Comprehensive Review of LLM Applications and Vulnerabilities," arXiv:2405.12750, 2024.
- [4] B. Wei, Y. S. Tay, H. Liu, J. Pan, K. Luo, Z. Zhu, and C. Jordan, "CORTEX: Collaborative LLM Agents for High-Stakes Alert Triage," arXiv:2510.00311, 2025.
- [5] Srinivas, Kirk, Zendejas, Espino, Boskovich, Bari, Dajani, and Alzahrani, "AI-Augmented SOC: A Survey of LLMs and Agents for Security Automation," *J. Cybersecur. Priv.*, vol. 5, no. 4, Art. 95, 2025.
- [6] Ismail, R. Kurnia, F. Widyatama, I. M. Wibawa, Z. A. Brata, Ukasyah, G. A. Nelistiani, and H. Kim, "Enhancing Security Operations Center: Wazuh Security Event Response with Retrieval-Augmented-Generation-Driven Copilot," *Sensors*, vol. 25, no. 3, Art. 870, 2025.
- [7] M. Silva, D. Pinto, J. Vitorino, J. Gonçalves, E. Maia, and I. Praça, "GeNIS: A Modular Dataset for Network Intrusion Detection and Classification," *Data in Brief*, 2025, doi:10.1016/j.dib.2025.111487.
- [8] S. Ndichu, T. Ban, S. Ozawa, T. Takahashi, and D. Inoue, "AI-Driven Security Alert Screening and Alert Fatigue Mitigation in Security Operations Centers: A Survey," arXiv:2605.08316, 2026.
- [9] Microsoft, "GUIDE: Microsoft Security Incident Prediction Dataset," Kaggle, 2024. [Online]. Available: <https://www.kaggle.com/datasets/Microsoft/microsoft-security-incident-prediction>
- [10] Confident AI, "DeepTeam: The Open-Source LLM Red Teaming Framework," GitHub repository, 2025. [Online]. Available: <https://github.com/confident-ai/deepteam>